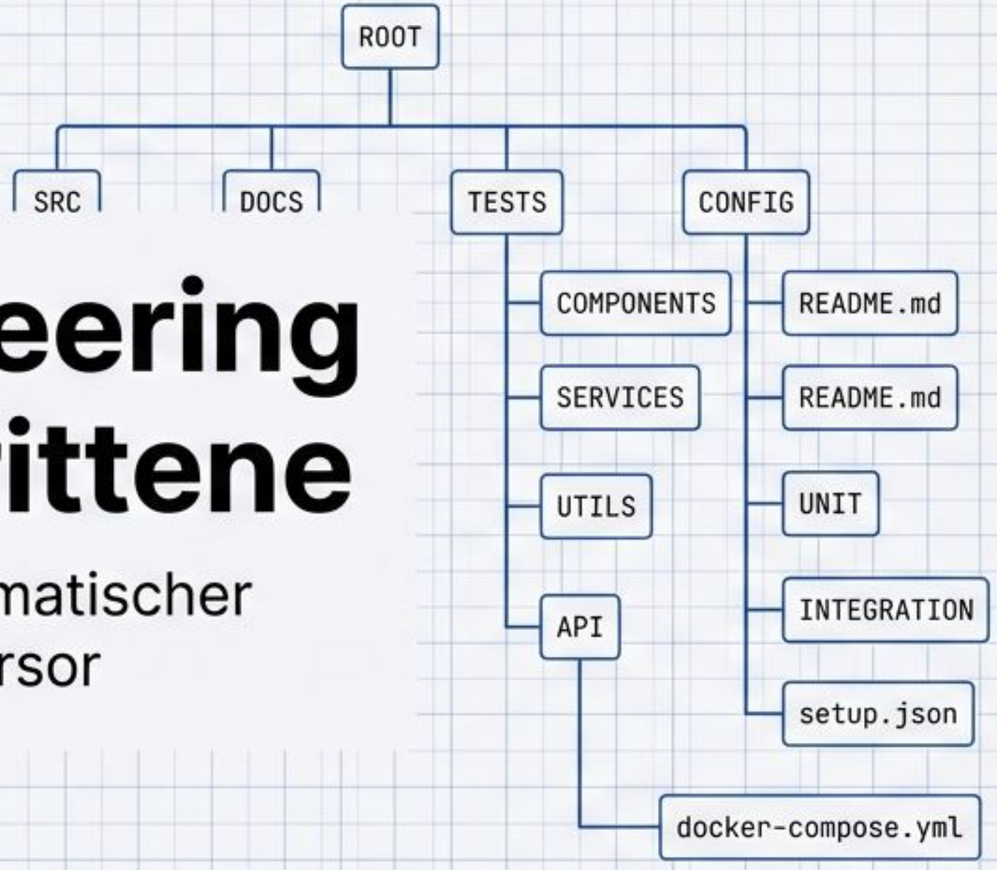
Vibe Coding (Level 2)

Context Engineering für Fortgeschrittene

Von “Vibe Coding” zu systematischer KI-Entwicklung mit Cursor

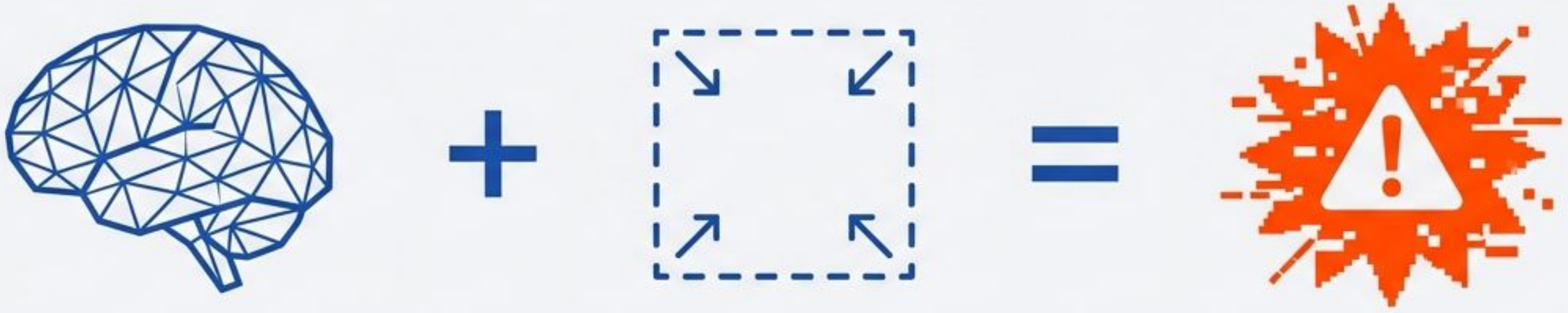
Context Engineering (Level 3+)



Workshop-Konzept für Senior Engineers

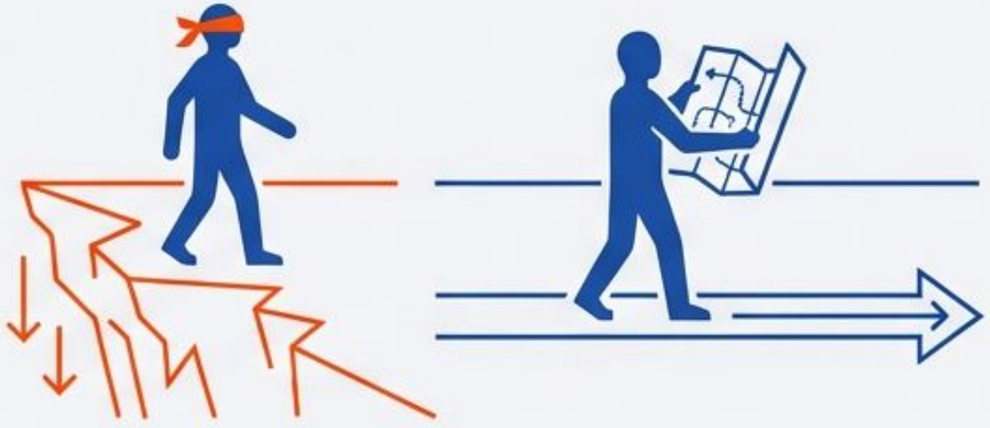
NotebookLM

Agent-Fehler sind keine Modell-Fehler. Sie sind Kontext-Fehler.



Hohe Intelligenz + Null Kontext = Halluzination

Wenn das Modell “halluziniert”, hat der Prompt keine Grenzen gesetzt. Wir behandeln KI oft wie einen allwissenden Senior-Partner, obwohl sie ohne Kontext wie ein brillanter neuer Mitarbeiter am ersten Tag ist: Voller Potenzial, aber ohne Wissen über das Projekt.



NotebookLM

Die 5+ Level der KI-Coding-Reife: Vom Tippen zum Steuern

Evolutionärer Pfad der KI-gestützten Softwareentwicklung und die Bedeutung von Context Engineering.

Die Stufen der KI-Evolution (Level 0–5)

Level 0 bis 2: Die Assistenz-Phase

Vom "Spicy Autocomplete" (L0) über einfache Aufgaben (L1) bis zum Pair Programming als Junior-Partner (L2).

Level 3 bis 4: Die Management-Phase

Entwickler agieren als Manager (L3) oder Product Manager (L4), die Specs schreiben und Agenten-Output steuern.

Level 5: Die "Dark Factory"

Volle Automatisierung – Spezifikationen führen direkt zu fertiger Software ohne menschliche Code-Reviews.

Die Realität am Arbeitsplatz

Die Level-2-Stagnation

90% der „KI-nativen" Entwickler verharren auf Level 2 und verpassen die fundamentale Transformation.

Das Produktivitäts-Paradoxon

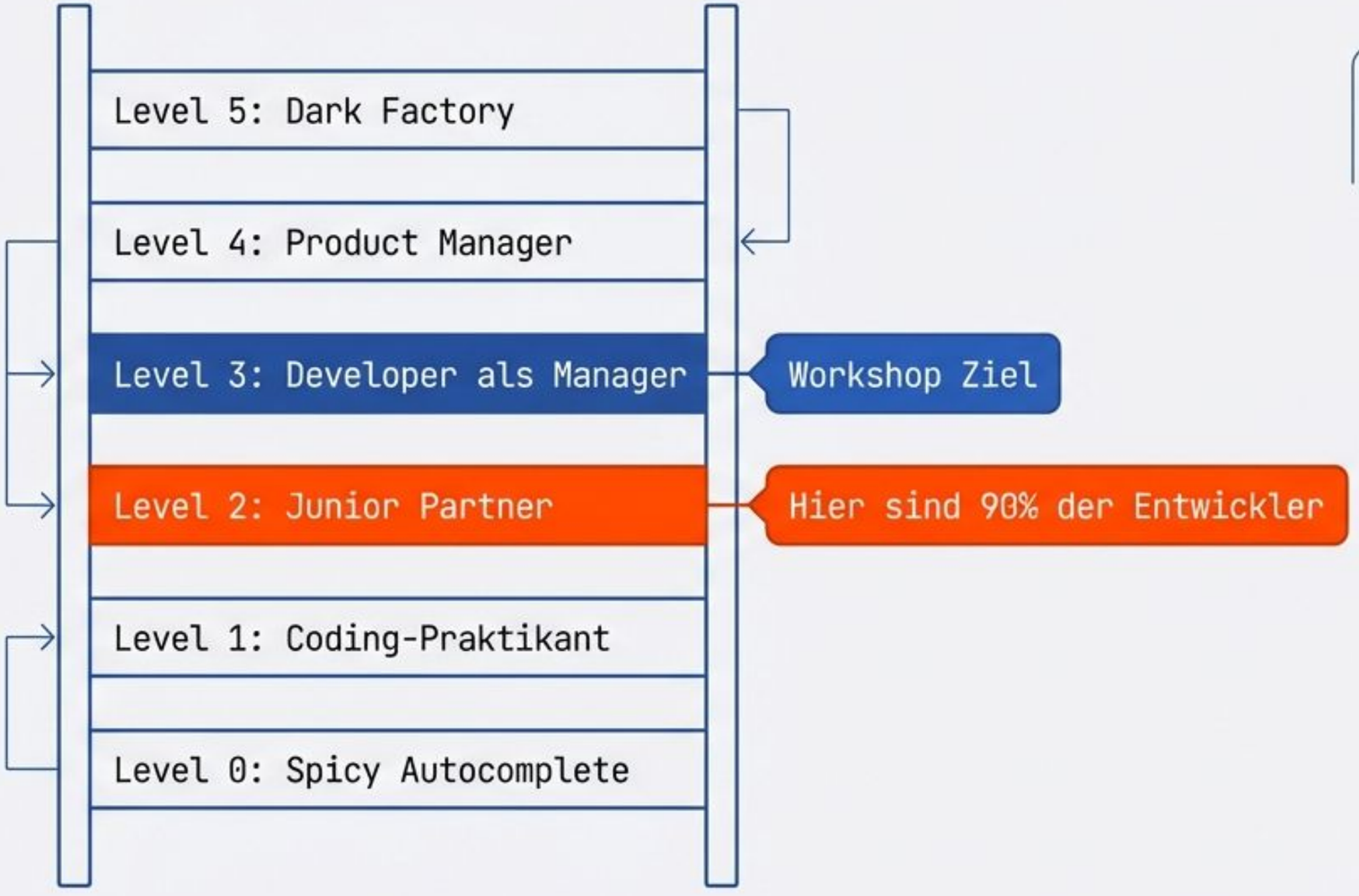
Entwickler fühlen sich oft 24% schneller, sind laut METR-Studie aber real 19% langsamer ohne Systematik.

Vergleich der Arbeitsweise (Status Quo vs. Transformed Future)

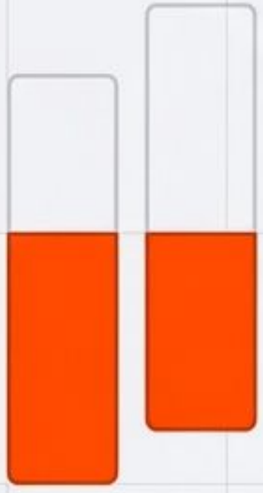
STATUS QUO (LEVEL 2)	TRANSFORMED FUTURE (LEVEL 3+)
Rolle: Code schreiben / korrigieren	**Rolle:** Spezifikationen & Context Design
KI-Modus: Chat / Tab-Completion	**KI-Modus:** Agent-Mode & Spec-Driven Dev
Workflow: Manuelles Iterieren	**Workflow:** Systematische Exploration

NotebookLM

Die Realitätslücke: Warum wir auf Level 2 feststecken



METR-Studie (2025)



Ohne systematischen Prozess erzeugt KI mehr Review-Aufwand als sie Zeit spart.

NotebookLM

Die Grenzen des 'Chat-Modus'

Drei Kernprobleme aus der Senior-Engineer-Umfrage

1. Vertrauen & Halluzination

"Code-Review ist zwingend." KI rät proprietäre Funktionen (SPL) oder ignoriert Ausschlüsse.

2. Workflow-Brüche

Das 'Copy-Paste-Ping-Pong". Medienbrüche zwischen Branch-Checkout und Prompt-Erstellung.

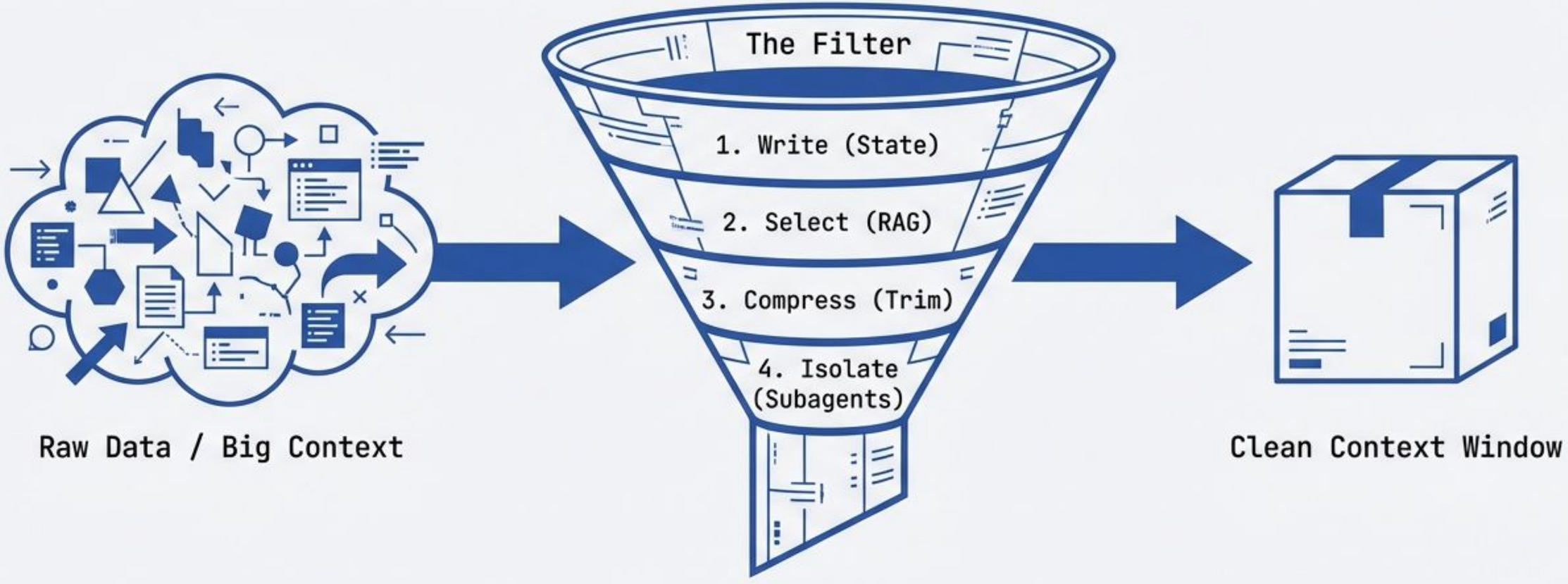
3. Context Rot (Vergesslichkeit)

Lange Chats führen zu n^2 Attention-Verlust. Das Modell 'vergisst' frühere Instruktionen, wenn die Token-Grenze erreicht ist.

NotebookLM

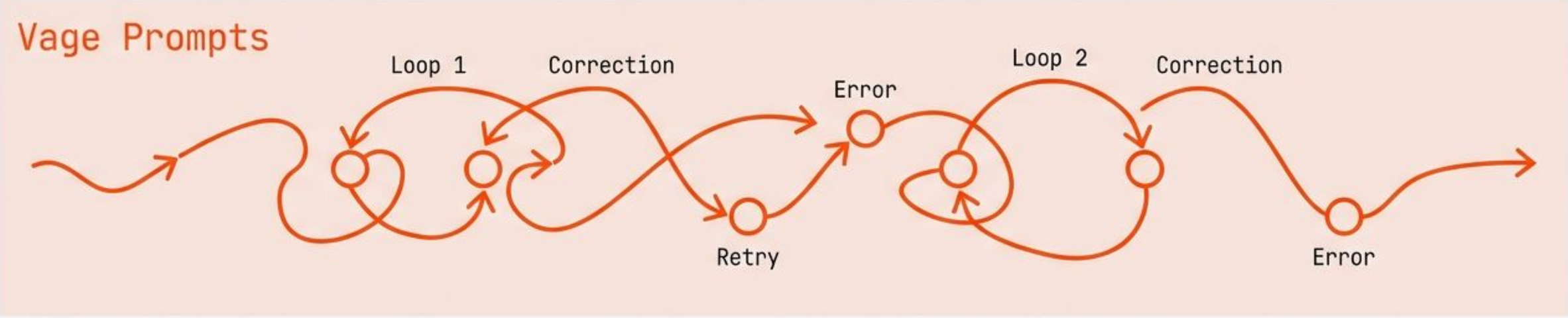
Context Engineering als neue Disziplin

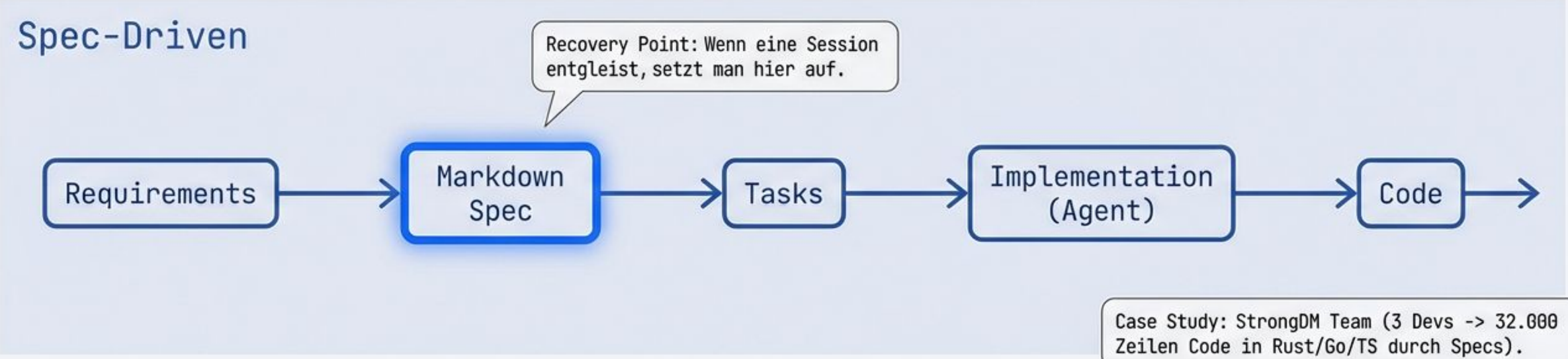
“Die delikate Kunst, das Context Window mit genau den richtigen Informationen für den nächsten Schritt zu füllen.” — Andrej Karpathy



NotebookLM

Erst die Spezifikation, dann der Code





Case Study: StrongDM Team (3 Devs -> 32.000 Zeilen Code in Rust/Go/TS durch Specs).

NotebookLM

Die Intelligenz kodifizieren: .cursor/rules

Von repetitiven Prompts zu permanenten Instruktionen (.mdc)

Frontmatter (Trigger-Logik)

```
---
description: SPL-Sprachreferenz
globs: ["**/*.spl"]
---

# SPL-Konventionen

- String-Konkatenation mit `&`, nicht `+`
- NIEMALS Python-Funktionen vorschlagen
- @spl-api-docs.md für Signaturen prüfen
```

Rule Body (Permanenter Kontext)

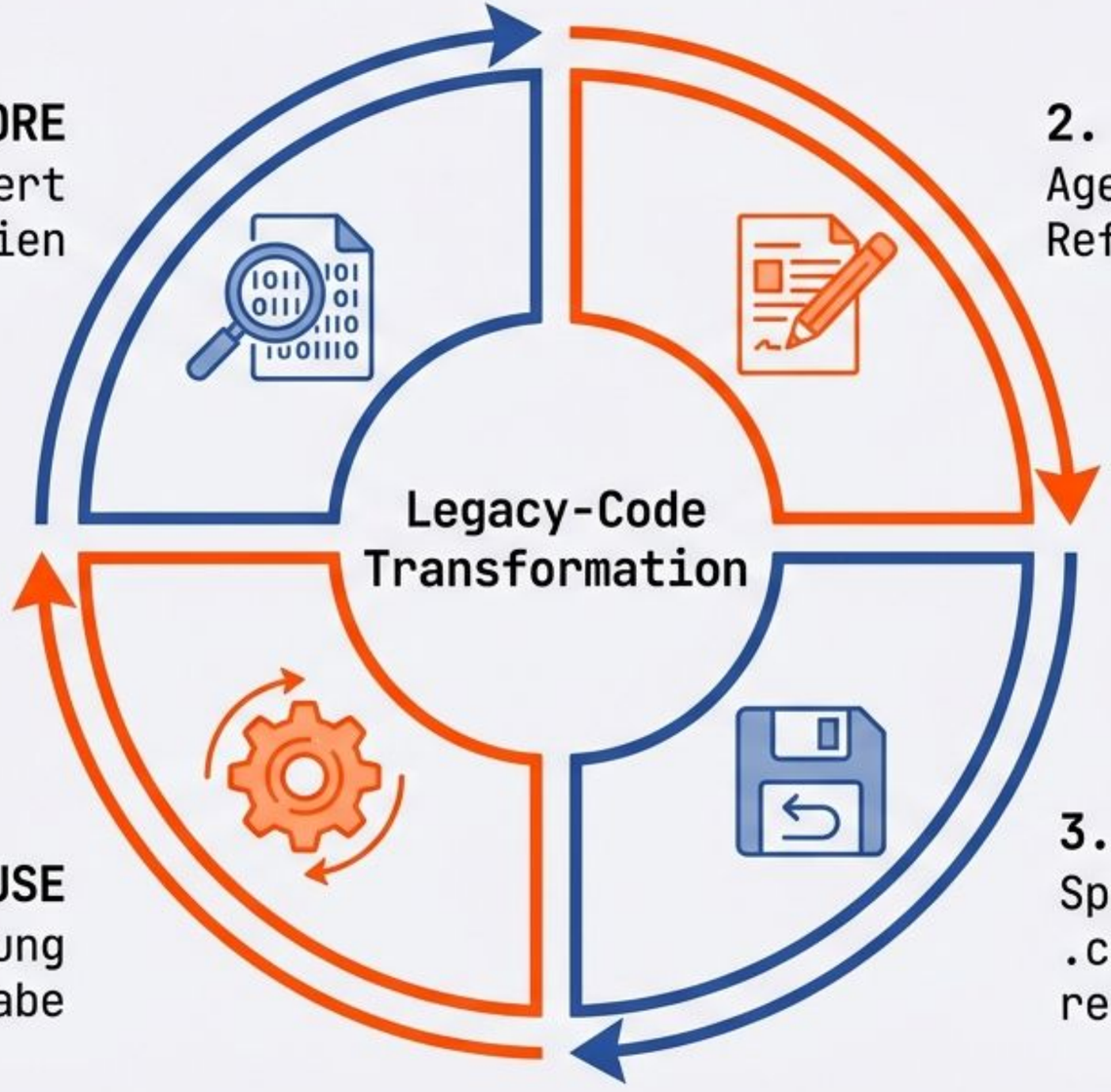
Anwendung: Rules definieren Tech-Stack, Testing-Patterns und 'Don't do X' Constraints automatisch basierend auf Dateitypen.

NotebookLM

Proprietary Code sichtbar machen (SPL & Interna)

1. EXPLORE
Agent analysiert unbekannte .spl Dateien

2. DOCUMENT
Agent erstellt Referenz-Markdown



3. PERSIST
Speichern als .cursor/rules/spl-reference.mdc

4. USE
Automatische Anwendung bei jeder SPL-Aufgabe

NotebookLM

Die Brücke zu internen Daten: MCP

Model Context Protocol

Standard: Docs-Integration.

Proprietary: Interne API-Doku oder DB-Schemas direkt im Chat verfügbar machen.

NotebookLM

Der richtige Modus für die richtige Aufgabe

AGENT MODE

Doing

Autonome Multi-File-Änderungen.

PLAN MODE

Thinking

Komplexe Architektur-Entscheidungen vor dem Code.

ASK MODE

Learning

Codebase verstehen ohne Änderungen.

YOLO MODE

Iterating

Rapid TDD Loops. Agent führt Terminal-Commands aus.

Vermeide Context Rot durch gezielten Modus-Wechsel.

NotebookLM

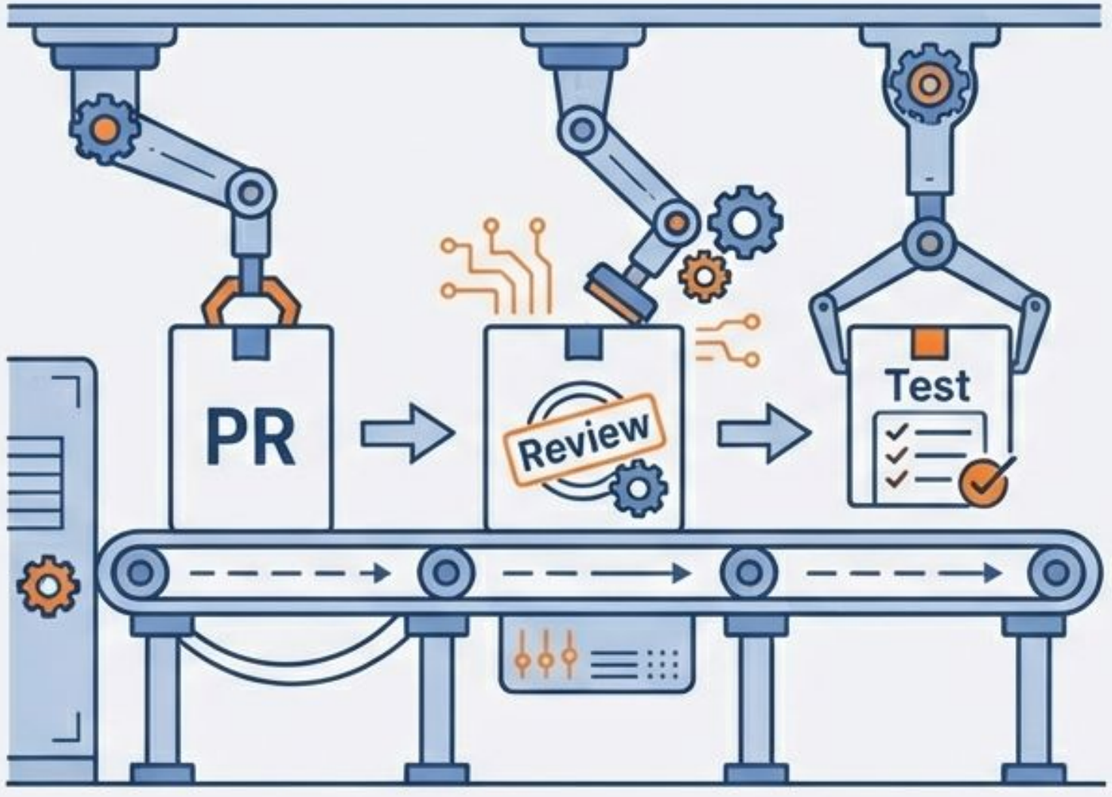
Tag 1: Bewusstsein & Grundlagen

'Be the Model' Exercise

- ☑ Das Erlebnis: Vage Prompts führen zu falschen Annahmen.
- ☑ Spec-Driven Development (Markdown).
- ☑ Cursor Rules & Indexing (.mdc).
- ☑ Proprietary Stack Setup (für SPL-Gruppe).

NotebookLM

Tag 2: Enterprise Workflows & Qualität



'Automatisierte CI/CD Pipeline'

- ☑ Testing Strategie: Plan -> Test -> Refactor.
- ☑ Multi-Repo Workflows.
- ☑ Automatisierte PRs & Reviews (/pr, /review).
- ☑ Background Agents für asynchrone Tasks.

NotebookLM

Das Ziel: Der Developer als Product Manager

Level 2 (Heute)	Code schreiben, Syntax checken, Prompting-Ping-Pong
Level 3+ (Ziel)	Specs definieren, Architektur bewerten, Qualität des Kontexts managen

Neue Metrik: Nicht ‘Lines of Code written’, sondern ‘Quality of Context provided’.

NotebookLM

Vom Vibe Coding zum Context Engineer: Dein Cursor Cheat Sheet

Die Transformation von Level 2 (Junior-Dev-Partner) zu Level 3+ (Developer als Manager) durch systematisches Kontext-Engineering.

Agent-Fehler sind keine Modell-Fehler — sie sind Kontext-Fehler.

Level 2: Der Junior-Dev-Partner (Status Quo)

Man nutzt die Ki ais Pair-Programmer im Flow State, verändert aber den grundlegenden Workflow nicht – 90% der Entwiekler stecken hier fest.

Level 3: Der Developer als Manager (Das Ziel)

Du schreibst keinen Code mehr selbst, sondern reviewst Diffs und steuerst mehrere Agent-Tabs parallel über präzise Spezifikationen.

Die Wahrnehmungsfalle

Laut METR-Studie fühlten sich Entwickler mit KI 24% schneller, waren aber tatsächlich 19% langsamer – systematischer Kontext ist die Lösung.

Workshop-Ziele vs. Cursor-Fähigkeiten Mapping

Workshop-Lernziel	Cursor-Funktion / Feature
Halluzinationen & Falschnahmen stoppen	**Cursor Rules (.cursor/rules/*.mdc):** Festlegen von Constraints und Projekt-Standards.
Proprietären Code (z.B. SPL) verstehen	**Codebase Indexing & @Docs:** Systematischer Aufbau von Referenz-Markdown-Datelen.
Kontext-Verlust (Context Rot) vermeiden	**Agent Mode & Neue Chats:** Isolierte Context Windows für spezifische Teilaufgaben nutzen.
Komplexe Architektur-Änderungen	**Spec-Driven Development:** Markdown-Specs im Plan Mode erstellen und vom Agent umsetzen lassen.
Brücke zu internen Datenquellen	**MCP (Model Contest Protocol):** Datenbanken, Wikis oder APIs direkt an den Agent anbinden.
Qualitätssicherung & Tests	**YOLO Mode + Test-Rules:** Iterative Test-Zyklen durch Agenten ohne manuelle Freigabe.

Der neue Workflow: Spec-Driven Development

Phase 1: Requirements & Design (Plan Mode)

Nutze Shift+Tab, um eine Markdown-Spezifikation zu entwerfen, bevor die erste Zeile Code generiert wird.

Phase 2: Implementation (Agent Mode)

Gib die fertige Spec als Kontext an den Agent Mode, um autonome Multi-File-Änderungen durchzuführen.

Phase 3: Validation (Debug Mode)

Nutze Agents mit Zugriff auf Runtime-Logs, um die Umsetzung direkt gegen die Spezifikation zu prüfen.

Quick-Ref: Welcher Modus für welche Aufgabe?

Ask Mode (Strg+L)

Ideal für „Read-only" Untersuchungen und das Verständnis der Codebase ohne Änderungen.

Agent Mode (Strg+I)

Der Standard für autonome Aufgaben, die Änderungen an mehreren Dateien erfordern.

Plan Mode

Beste Wahl für die strategische Planung komplexer Features vor der Umsetzung.

YOLO Mode

Aktiviert autonome Terminal-Befehle für schnelle TDD-Zyklen (Test Driven Development).

NotebookLM